

Will AI Programming Replace All Programmers? A Look at What the Industry Actually Shows

Zhihu · 2026-09-19

Since AI programming improved dramatically in late 2025 and early 2026, a large part — in many cases almost all — of programmers' coding work has been taken over by AI. This phenomenon has spread panic and despair among programmers. At the same time, AI companies have used the advances in this field to inflate AI's capacities without limit, claiming that within a year or two all white-collar workers will be laid off. Every kind of self-media outlet has piled in, spreading panic and manufacturing anxiety.

We need to examine this question seriously. Can AI programming replace all programmers? If not, what is the actual replacement rate?

In fact, in an answer I wrote a few months ago (AI's First Strike Falls on Programmers — Are Programmers Facing Mass Unemployment?), I gave my own judgement: AI cannot replace all human labour, and the end-to-end replacement rate is no higher than 30%.

Once AI-centred intelligent machine systems enter production on a large scale, manual coding work will basically disappear. Of course, the development of core code that is architecturally complex or subject to exceptionally high performance requirements may still need heavy human involvement, but the great mass of ordinary code development will be wholly taken over by AI. How many people will actually be replaced? Judging by my own practical experience and by others' reports on Zhihu, the productivity gain is roughly 20% to 50%; if AI coding lands properly (it is still in large-scale industry experimentation), and assuming that pure developers are 60% of the total, that means cutting roughly 12% to 30% of programmer posts.

AI coding has been landing on a large scale for at least half a year now, especially in China and the US, where the big firms basically require their employees to use AI agents for development. So what has the effect actually been?

According to a McKinsey report^[1], for programmers who "know how to use AI", productivity can be raised by roughly 16%–30%, broadly consistent with my earlier judgement.

 Figure 1

More recent research^[2] also shows that although AI programming greatly speeds up code generation, when it comes to the final product release, the efficiency gain is only about 30%. This, again, is

consistent with the McKinsey research above and with my earlier estimate.

Generative AI tools for software development have advanced at astonishing speed, evolving in just a few years from simple code autocompletion to fully autonomous coding agents. Can these gains in capability be translated into task-level productivity growth in the real world? And even if they can, does higher efficiency at the task level mean higher final output? These questions are crucial to understanding the economic impact of AI. Using a matched event study of developers, we find that as AI coding tools iterate and improve, their task-level productivity effects become more and more pronounced: autocompletion raised weekly code commits by about 30%, synchronous agents lifted this cumulative effect to about 180%, and asynchronous agents to about 240%. Yet as we move up the hierarchy to the production stage, these gains fall off sharply: although synchronous agents increased lines of code by 958% and pull requests by 86%, the number of final releases grew by only 20%. Overall coding activity on GitHub shows the same pattern.

 Figure 2

Source: <https://mitsloan.mit.edu/ideas-made-to-matter/ai-boosts-worker-productivity-does-translate-to-final-outputs>

Not only are various studies confirming the limits of AI programming; the developers and managers who actually use AI agents feel it even more directly. Programmers' attitude to AI programming has sobered up from the initial shock, and through practical production use they have gradually come to recognise AI's limits. Cal Newport (a professor of computer science at Georgetown University), for instance, in a blog post^[3] discusses how a senior Silicon Valley programmer's attitude turned around over the past six months. He started using Claude Code early in the year. After programming with Claude Code, his productivity rose several-fold and he exclaimed "really, really good"; but after two online incidents caused by AI code, in which he nearly lost his job, he became disillusioned with AI programming, began writing core code by hand, and used AI programming only for code such as test cases and scripts. This developer holds that programming the "old-school" way is slow but reliable, with guaranteed quality. The author notes that this situation is not isolated, and that quite a few engineers in the industry are expressing similar views.

Where does AI programming go wrong? One programmer, in a blog post^[4] talks about his impressions from six months of reviewing AI code. In his view, an AI agent is like a very capable new employee: the code it writes looks good, but because it lacks domain background knowledge, it is always careless about the various constraints, boundaries, and exceptional scenarios, and it readily introduces security problems. In fact, a recent paper, Constraint Decay: The Fragility of LLM Agents in Backend Code Generation^[5], also discusses this problem:

Our study reveals a phenomenon of "constraint decay": as structural requirements accumulate, agent performance drops markedly. From benchmark tasks to fully specified tasks, better-performing

configurations fell on average by 30 percentage points in assertion pass rates, while some weaker configurations dropped to near zero. A sensitivity analysis across frameworks revealed significant performance differences: agents perform well in minimalist, explicit frameworks such as Flask, but in environments that rely heavily on convention, such as FastAPI and Django, average performance falls sharply. Finally, error analysis shows that defects at the data layer (for example, wrongly constructed queries and ORM runtime exceptions) are the main source of these problems. This study stresses that satisfying functional and structural requirements at the same time remains a key and still unsolved challenge for coding agents.

A CTO at an Indian enterprise also mentions in a recent article^[6] that AI-written code easily runs into problems in actual production because "in large systems, code must conform to architectural patterns, dependency chains, business rules, compliance requirements, security controls, and release processes. If AI-generated code ignores these constraints, it shifts the focus of the work from development to subsequent repair and remediation, which in turn lengthens code review cycles, leaves test coverage insufficient, allows security risks to go undetected, causes integration failures, and produces defects in the production environment." Anyone with practical experience of AI programming will understand that these problems are indeed widespread.

AI code is written fast, but because of the problems described above, developers must scrutinise AI-generated code carefully before it can be merged into the repository. This gives rise to two problems. First, the speed of code review is markedly lower than the speed of code generation, so the productivity gain from AI is much reduced. Second, new programmers lack the opportunity to write code with their own hands and cannot improve their skills; once the older generation of programmers is laid off or retires, code review will in the end become even slower, and reviewers may fail to spot problems at all.

What is more, some enterprises place great hopes on AI programming and drastically compress development cycles, leaving programmers no time at all to review code carefully. To the developer, the enterprise is not his own; when the boss demands speed, he reviews cursorily or merges without reviewing at all. If something goes wrong after the release, the worst that happens is that he leaves. Whether from review fatigue or from insufficient review, the end result is that the code entering the production environment harbours a great heap of bugs, code smells and similar problems, while the people who maintain that software are in fact unfamiliar with the code. All this technical debt produces what is called "The Maintainability Gap"^[7].

AI-assisted programming continues to advance rapidly, and the most enthusiastic developers are producing historically high volumes of code. Yet analysis of four years of code-change data shows that code maintainability indicators are declining: cross-file function calls (which reflect code reuse) fell by 35%; compared with 2022 levels, lines migrated in refactoring fell by 70%, and maintenance work on

long-standing legacy code fell by 74%. At the same time we observe a worrying rise in several phenomena: "copy/paste" behaviour within a single commit rose by 41%, duplicated code blocks by 81%, error-concealing coding structures by 47%, and two-week code churn by 15%. Code output has indeed risen substantially, but the technical debt accumulating as a result cannot be ignored; this debt is concentrated among those developers who have not yet realised that certain coding patterns endanger the long-term maintainability of the codebase.

On technical debt, the paper *Debt Behind the AI Boom*^[8], through a large-scale study, shows that "code smells are the most common type of problem, accounting for as much as 89.3%"; "in commits produced by every kind of AI programming assistant, more than 15% introduced at least one problem. More important, of the problems introduced by AI, 22.7% still remain in the latest version of the code repository." "These findings suggest that AI-generated code may impose long-term maintenance costs on real software projects, and they also highlight the need to strengthen quality assurance in AI-assisted development."

 Figure 3

From: *Debt Behind the AI Boom*

While the paper *Faster Code, Deeper Debt?*^[9] points out that "LLMs tend to aggravate traditional forms of technical debt (especially debt in code, design and documentation), while also introducing new debt specific to LLMs. Notably, we identify a kind of 'rapid integration debt': sacrificing quality in pursuit of speed, so that rapidly generated code triggers a chain reaction, producing governance debt and raising long-term maintenance costs. Other emerging categories of debt include prompt debt, ethical debt, data debt and provenance debt, which reflect the distinctive challenges brought by LLM applications."

AI programming not only leads to the accumulation of technical debt; it also introduces other kinds of debt. The computer scientist Margaret-Anne Storey, in her paper *From Technical Debt to Cognitive and Intent Debt*^[10] proposes that, beyond technical debt, "as the speed of AI-generated code exceeds the speed at which the team understands it, two often-underestimated forms of 'debt' are accumulating: first, "cognitive debt", that is, the loss of shared understanding within a team; second, "intent debt", that is, the lack of the externalised logic and rationale that developers and AI agents need in order to handle code safely." In plain language, the comprehensive use of AI programming may leave a whole team with no one who clearly knows what they are doing. At present the industry's attention is still fixed on technical debt, and these deeper problems have not yet been fully exposed.

At this point I think my readers already have their answer. In many answers before now (such as: *Do You Think AI Programming Will Really Replace Programmers in the Future?* and *How Should We*

<https://calnewport.com/on-ai-coding-and-its-discontents/>

[4] I Spent 6 Months Reviewing AI Code. Here's the Ugly Truth

中文原文: I Spent 6 Months Reviewing AI Code. Here's the Ugly Truth

https://medium.com/@haseeb_sohail/i-spent-6-months-reviewing-ai-code-heres-the-ugly-truth-b81fe33e0b25

[5] Constraint Decay: The Fragility of LLM Agents in Backend Code Generation

中文原文: Constraint Decay: The Fragility of LLM Agents in Backend Code Generation

<https://arxiv.org/pdf/2605.06445>

[6] The Productivity Illusion at the Heart of Enterprise AI Coding

中文原文: The Productivity Illusion at the Heart of Enterprise AI Coding

<https://cacm.acm.org/blogcacm/the-productivity-illusion-at-the-heart-of-enterprise-ai-coding/>

[7] The Maintainability Gap: AI Code Quality in 2026

中文原文: The Maintainability Gap: AI Code Quality in 2026

https://www.gitclear.com/the_ai_code_quality_maintainability_gap

[8] Debt Behind the AI Boom: A Large-Scale Empirical Study of AI-Generated Code in the Wild

中文原文: Debt Behind the AI Boom: A Large-Scale Empirical Study of AI-Generated Code in the Wild

<https://arxiv.org/abs/2603.28592>

[9] Faster Code, Deeper Debt? A Multivocal Literature Review on Technical Debt and Its Early Signs in LLM-Assisted Software Development

中文原文: Faster Code, Deeper Debt? A Multivocal Literature Review on Technical Debt and Its Early Signs in LLM-Assisted Software Development

<https://arxiv.org/abs/2606.14796>

[10] From Technical Debt to Cognitive and Intent Debt: Rethinking Software Health in the Age of AI

中文原文: From Technical Debt to Cognitive and Intent Debt: Rethinking Software Health in the Age of AI

<https://arxiv.org/abs/2603.22106>

This article was originally published on Zhihu (in Chinese): <https://zhuanlan.zhihu.com/p/2084561490930045240> Author: 最最遥远的路 · Zhihu profile: <https://www.zhihu.com/people/zui-zui-yao-yuan-de-lu-32/posts>

本文原发表于知乎专栏/回答 (中文): <https://zhuanlan.zhihu.com/p/2084561490930045240>; 作者: 最最遥远的路: <https://www.zhihu.com/people/zui-zui-yao-yuan-de-lu-32/posts>